

NumPy Practice Questions

From Absolute Basics to Intermediate Level

This workbook contains a graded set of practice problems for learning the NumPy library, organized into five progressive sections. Work through them in order — each section builds on skills from the one before it. Try to solve every question in a Python/Jupyter environment before checking any external references.

Section	Topic	Level	# Qs
1	Arrays: Creation & Attributes	Very Basic	15
2	Indexing, Slicing & Reshaping	Basic	15
3	Arithmetic, Broadcasting & Math Functions	Basic-Intermediate	18
4	Aggregation, Boolean Masking & Sorting	Intermediate	16
5	Linear Algebra, Random & Advanced Ops	Intermediate	16

Total: 80 questions | Assumes: `import numpy as np`

Section 1: Arrays — Creation & Attributes [Very Basic]

Creating Arrays

- 1.1 Import NumPy under the alias **np** and print its version.
- 1.2 Create a 1-D array from the Python list **[1, 2, 3, 4, 5]**.
- 1.3 Create a 2-D array (3 rows, 3 columns) containing the numbers 1 to 9.
- 1.4 Create an array of 10 zeros.
- 1.5 Create an array of 10 ones, with data type **int32**.
- 1.6 Create a 3x3 identity matrix using **np.eye()**.
- 1.7 Create an array of 5 elements, all set to the value 7, using **np.full()**.
- 1.8 Create an array of integers from 0 to 20 using **np.arange()**, stepping by 2.
- 1.9 Create an array of 10 evenly spaced numbers between 0 and 1 using **np.linspace()**.
- 1.10 Create an empty (uninitialized) array of shape (2, 3) using **np.empty()**.

Array Attributes

- 1.11 For a given array **a**, print its shape, size, and number of dimensions (**ndim**).
- 1.12 Check and print the data type (**dtype**) of an array of floats.
- 1.13 Create an array of floats and convert it to integers using **.astype()**.
- 1.14 Find the number of bytes each element occupies in an array (**itemsize**).
- 1.15 Create a 1-D array of 12 elements and determine what shapes it could be reshaped into without changing the total number of elements.

Section 2: Indexing, Slicing & Reshaping [Basic]

Indexing & Slicing

- 2.1 Given `a = np.array([10, 20, 30, 40, 50])`, retrieve the third element.
- 2.2 From the same array, retrieve the last element without using its index number.
- 2.3 Slice out elements from index 1 to 3 (inclusive) of a 1-D array.
- 2.4 Reverse a 1-D array using slicing.
- 2.5 Given a 2-D array of shape (4, 4), extract the second row.
- 2.6 From the same 2-D array, extract the third column.
- 2.7 Extract a 2x2 sub-matrix from the top-left corner of a 4x4 array.
- 2.8 Use step slicing to select every other element of a 1-D array.
- 2.9 Given a 2-D array, select all rows but only the first two columns.
- 2.10 Replace all elements greater than 15 in an array with the value 0, using boolean indexing.

Reshaping & Combining

- 2.11 Reshape a 1-D array of 12 elements into a (3, 4) 2-D array.
- 2.12 Flatten a 2-D array back into a 1-D array using `.flatten()` and `.ravel()` — explain the difference in behavior.
- 2.13 Stack two 1-D arrays vertically using `np.vstack()`.
- 2.14 Stack two 1-D arrays horizontally using `np.hstack()`.
- 2.15 Split an array of 9 elements into 3 equal parts using `np.split()`.

Section 3: Arithmetic, Broadcasting & Math Functions

[Basic-Intermediate]

Arithmetic Operations

- 3.1 Add two arrays of the same shape element-wise.
- 3.2 Multiply an array by a scalar (e.g., multiply every element by 3).
- 3.3 Compute the element-wise square of every value in an array.
- 3.4 Compute the element-wise square root of an array using **np.sqrt()**.
- 3.5 Divide two arrays element-wise and handle a potential division-by-zero case.
- 3.6 Compute the dot product of two 1-D arrays using **np.dot()**.

Broadcasting

- 3.7 Add a 1-D array of shape (3,) to a 2-D array of shape (3, 3) and explain what broadcasting rule allows this.
- 3.8 Given a (4, 1) array and a (1, 4) array, add them together and describe the resulting shape.
- 3.9 Normalize the rows of a 2-D array by subtracting each row's mean and dividing by its standard deviation, using broadcasting (no loops).
- 3.10 Explain (in code and comments) why an array of shape (3, 4) cannot be broadcast with one of shape (3,) without reshaping, and show the fix.

Mathematical & Statistical Functions

- 3.11 Compute the exponential (**np.exp()**) of each element in an array.
- 3.12 Compute the natural log (**np.log()**) of each element in an array of positive numbers.
- 3.13 Round every element of a float array to 2 decimal places using **np.round()**.
- 3.14 Compute the absolute value of every element in an array containing negative numbers.
- 3.15 Compute sine and cosine of an array of angles (in radians) using **np.sin()** and **np.cos()**.
- 3.16 Clip all values in an array so they lie between 0 and 100 using **np.clip()**.
- 3.17 Compute the cumulative sum of a 1-D array using **np.cumsum()**.
- 3.18 Compute the cumulative product of a 1-D array using **np.cumprod()**.

Section 4: Aggregation, Boolean Masking & Sorting

[Intermediate]

Aggregation Functions

- 4.1 Compute the sum, mean, minimum, and maximum of a 2-D array — once across all elements, once per column, and once per row.
- 4.2 Find the index of the maximum value in a 1-D array using **np.argmax()**.
- 4.3 Find the index of the minimum value in a 2-D array (flattened) using **np.argmin()**.
- 4.4 Compute the standard deviation and variance of an array.
- 4.5 Compute the median of an array with an even number of elements.
- 4.6 Count how many elements in an array satisfy a condition (e.g., are greater than 50) using **np.sum()** on a boolean mask.

Boolean Masking & Filtering

- 4.7 Given an array of exam scores, create a boolean mask for scores greater than or equal to 60 (passing).
- 4.8 Use a boolean mask to extract only the even numbers from an array.
- 4.9 Use **np.where()** to replace negative values in an array with 0 and keep positive values unchanged.
- 4.10 Use **np.where()** to create a new array labeling each element as 'Pass' or 'Fail' based on whether it is ≥ 50 (hint: work with numeric codes since NumPy arrays are typed).
- 4.11 Combine two boolean conditions with **&** and **|** to filter values that are both greater than 10 and less than 50.
- 4.12 Use **np.isnan()** to identify and remove NaN values from an array.

Sorting & Unique Values

- 4.13 Sort a 1-D array in ascending order using **np.sort()**.
- 4.14 Sort a 2-D array along each row and separately along each column.
- 4.15 Get the indices that would sort an array using **np.argsort()**, without modifying the original array.
- 4.16 Find the unique values in an array, along with their counts, using **np.unique()**.

Section 5: Linear Algebra, Random & Advanced Ops

[Intermediate]

Linear Algebra

- 5.1 Compute the matrix product of two 2-D arrays using **np.matmul()** or the **@** operator.
- 5.2 Compute the transpose of a 2-D array using **.T**.
- 5.3 Compute the determinant of a 3x3 matrix using **np.linalg.det()**.
- 5.4 Compute the inverse of a square, invertible matrix using **np.linalg.inv()**.
- 5.5 Solve a system of linear equations $Ax = b$ using **np.linalg.solve()**.
- 5.6 Compute the eigenvalues and eigenvectors of a square matrix using **np.linalg.eig()**.

Random Number Generation

- 5.7 Generate a 1-D array of 5 random floats between 0 and 1 using **np.random.rand()**.
- 5.8 Generate a (3, 3) array of random integers between 10 and 50 using **np.random.randint()**.
- 5.9 Set a random seed with **np.random.seed()** and show that it produces reproducible results.
- 5.10 Randomly shuffle the rows of a 2-D array using **np.random.shuffle()**.
- 5.11 Randomly sample 3 elements from a 1-D array without replacement using **np.random.choice()**.

Advanced Operations

- 5.12 Use **np.concatenate()** to join three 1-D arrays into one.
- 5.13 Add a new axis to a 1-D array to convert it into a 2-D column vector, using **np.newaxis** or **.reshape()**.
- 5.14 Use **np.tile()** to repeat a small array to build a larger tiled pattern.
- 5.15 Use vectorized operations (no explicit Python loop) to compute the pairwise Euclidean distance between two sets of points stored as 2-D arrays.
- 5.16 Given a large array, measure and compare the execution time of a vectorized NumPy operation versus an equivalent pure-Python **for** loop doing the same computation.

Tip: For each question, try to solve it in a single line using vectorized NumPy operations before resorting to loops — this is the core skill NumPy practice builds.